Comparison: ACDCs and Merkle Tree Certificates

Daniel Hardman

2026-01-06

Abstract

This paper compares ACDCs and Merkle Tree Certificates, analyzing their design, use cases, and implications for digital trust.

1. Introduction and scope

Two different communities have been circling a similar problem from opposite directions.

On one side are identity and credential systems that have grown uneasy with the way trust is handled on the modern internet. Static identifiers, brittle revocation mechanisms, and routine over-disclosure all make it harder to build systems that respect autonomy and privacy without sacrificing verifiability. KERI and ACDCs emerged from this unease, with an emphasis on explicit key state, durable provenance, and the ability to reveal only what a given interaction requires [1, 2].

On the other side is the WebPKI, an infrastructure that has scaled astonishingly well but is showing signs of strain. Certificate transparency was bolted on to address mis-issuance after the fact [3]. Revocation remains unreliable in practice. And the prospect of post-quantum cryptography threatens to turn already heavy handshakes and logs into something much worse [4]. Merkle Tree Certificates are one attempt to relieve that pressure without tearing out the foundations of the web [5].

At first glance, these efforts seem only loosely related. One lives in the world of self-sovereign identity and verifiable credentials. The other is firmly rooted in browsers, certificate authorities, and TLS. Yet both rely on hash-based commitments arranged in Merkle-like structures. That shared vocabulary invites comparison, but it also creates room for misinterpretation.

This paper aims to clarify what that resemblance does and does not imply. It is not a bake-off. It does not argue that one system should replace the other, nor that they solve the same problem equally well. Instead, it treats each as a response to a different set of constraints, using some shared primitives in different ways.

The comparison is organized by function rather than lineage. Instead of asking which protocol came first, or which standards body is involved, we ask a simpler set of questions. What is being committed to, and why. What is being proven to a verifier, and what is left implicit. How freshness and revocation are handled in practice. Where the operational costs land. And how post-quantum pressures shape design choices.

The intended reader is technically literate, but not necessarily fluent in both worlds. If you are comfortable with certificate transparency and Merkle inclusion proofs, this paper aims to make ACDC graduated disclosure feel intelligible rather than exotic. If you are steeped in KERI and self-addressing identifiers, it aims to make Merkle Tree Certificates feel less like a curiosity and more like a coherent response to WebPKI realities.

The goal is modest but practical. By the end, a reader who knows one model well should be able to reason about the other without relying on analogies that do not quite fit.

2. A comparative map

Before going further, it helps to see the whole landscape at once.

The table below is not a summary of conclusions. It is a map. Each row corresponds to a question that any trust system must answer in practice. Each column shows how KERI/ACDCs and Merkle Tree Certificates answer that question, given their respective goals and constraints [1, 2, 5].

The sections that follow expand on these rows one by one. Readers who already know one side well may find it useful to treat the table as an index, returning to it as a reference point while reading the prose.

Axis	KERI / ACDCs	Merkle Tree Certificates (MTC)
Primary problem being optimized	Durable attribution, continuity of control, and privacy-preserving exchange of structured claims	Scalable, auditable certificate issuance under bandwidth, latency, and post-quantum pressure
Core commitment object	SAID: a self-addressing identifier that commits to the content and structure of a claim	Merkle roots and landmarks that commit to an append-only log of issuance events
What is being committed to	Meaning: the identity of a structured claim, independent of presentation	Membership: inclusion of an object in a publicly auditable history
What is proven to the verifier	That the disclosed view corresponds to a specific underlying claim and verifies equivalently	That a certificate was issued and logged as part of a committed set
Partial visibility mechanism	Graduated disclosure: substructures may be replaced by their SAIDs without changing verification semantics	Inclusion proofs: Merkle paths prove membership without revealing the rest of the log
Primary motivation for partiality	Privacy and correlation minimization	Efficiency and scalability

Axis	KERI / ACDCs	Merkle Tree Certificates (MTC)
Freshness assumptions	Emphasis on continuity and completeness of key state; verification can be offline	Emphasis on recency; efficiency depends on clients being up to date on landmarks
Revocation model	Expressed as events in key state or registries, reconciled over time	Rooted in X.509 semantics; transparency improves detection rather than enforcement
Operational cost center	Managing and reasoning about event history and state reconciliation	Distributing and caching landmarks at scale
Post-quantum pressure addressed	Algorithm agility and continuity across key transitions	Signature and certificate size on the wire and in logs
Standardization trajectory	ToIP KSWG specifications, ISO 17442-3, GSMA deployments, ecosystem layering [6, 7, 8]	IETF Internet-Draft, CT lineage, browser and CA experimentation [3, 5, 9]
Typical deployment surface	Identity systems, credentials, registries, telecom signaling	WebPKI, TLS handshakes, browser trust infrastructure

The resemblance between the two systems is most visible at the level of shared primitives. The sections that follow focus on how those primitives are used, and to what ends.

3. Shared primitives, different design goals

Both KERI/ACDCs and Merkle Tree Certificates rely heavily on hash commitments. Both use those commitments to make partial information verifiable. And both emerged in response to real stress in existing systems rather than as academic exercises [1, 3, 5].

These similarities explain why engineers from one world often feel an immediate sense of familiarity when encountering the other. They also explain why superficial comparisons can be misleading. The shared primitive sits at different layers in each system and is pressed into service for different ends.

3.1 Hash commitments as a common foundation

At the lowest level, both designs rely on standard cryptographic assumptions about hash functions: collision resistance, second-preimage resistance, and practical irreversibility [10]. In both cases, a hash acts as a durable stand-in for some larger body of data.

In ACDCs, the hash is elevated into the data model. A self-addressing identifier is not merely a checksum; it is a first-class field that other structures point to. Once that happens, the question of what an object commits to is answered structurally, not procedurally [2].

In Merkle Tree Certificates, the hash sits at the boundary between objects. Certificates remain conventional X.509 artifacts [11]. The hash-based structure wraps around them, organizing issuance events into an append-only tree. Here, the hash commits not to meaning but to position: this certificate appeared at a particular point in a public history [5].

The same mathematical tool is doing different kinds of work.

3.2 Trees, structure, and meaning

This difference shows up clearly in how tree structure is used.

In MTCs, the tree is literal. Leaves correspond to issued certificates. Internal nodes summarize subsets of issuance events. The root, or landmark, stands for a snapshot of the log at a particular moment. Traversing the tree is about proving membership and consistency [3, 5].

In ACDCs, the "tree" is implicit in the schema. Nested maps and lists form a semantic structure: attributes, assertions, relationships. Hashing that structure produces a graph of commitments whose shape reflects meaning rather than chronology [2].

A Merkle inclusion proof establishes placement in a history. A SAID establishes identity of content.

3.3 Different pressures, different responses

KERI and ACDCs grew out of a concern with attribution, continuity of control, privacy, and the cacheability of foundational claims that are referenced repeatedly. They respond to over-sharing and redundantly proving: revealing more than is necessary to establish trust, creating linkages that persist longer than intended, and not recognizing that proof has already been evaluated before [1, 2].

Merkle Tree Certificates grew out of a concern with scale and auditability in the WebPKI. The pressure there is accumulation: ever-larger certificates, ever-more signatures, and ever-growing logs, especially in a post-quantum context [4, 5].

Both systems reach for hashes because hashes are cheap, stable, and well understood. They deploy them to relieve different kinds of strain.

4. What is proven, to whom

At a glance, ACDC graduated disclosure and Merkle Tree Certificate inclusion proofs can look similar. Both rely on hashes. Both allow a verifier to check something without seeing everything. Beyond that surface similarity, the proof obligations diverge.

4.1 Graduated disclosure in ACDCs

An ACDC is a structured container whose contents are bound together by self-addressing identifiers. Each block of data is hashed, and that hash becomes part of the structure itself. Because parent blocks incorporate the hashes of their children, a verifier can check the integrity of the whole even if some of the parts are missing [2].

Graduated disclosure takes advantage of this structure. A presenter can replace selected substructures with their SAIDs, revealing only what a particular verifier needs to see. A fully expanded ACDC and a partially disclosed one verify the same way, lead to the same status checks, and anchor to the same underlying claim [2, 12].

This mechanism is designed to manage correlation risk. Different verifiers can see different views of the same claim without forcing over-disclosure. The hash commitments preserve semantic continuity across those views.

The proof obligation is semantic. The presenter is establishing that a particular view corresponds to a specific underlying claim.

4.2 Inclusion proofs in Merkle Tree Certificates

Merkle Tree Certificates address a different set of questions. In the WebPKI, certificates are public artifacts. The dominant concerns are correct issuance, append-only logging, and detectability of mis-issuance [3, 5].

An inclusion proof answers one narrow question: is this object a member of a committed set. In MTCs, that set is a subtree of a certificate issuance log, summarized by a landmark hash. A relying party that has the relevant landmark can verify, via a Merkle path, that a particular TBSCertificate is included in that subtree [3, 5].

Here, what is omitted is not sensitive structure but the rest of the log. The omission serves efficiency and scalability. The proof obligation is historical: the relying party is being assured that the certificate exists in a publicly auditable history that has not been rewritten.

4.3 Two distinct proof obligations

Graduated disclosure answers the question of how much of a claim must be revealed to establish trust. Inclusion proofs answer the question of whether a claim was issued and logged as asserted.

Both rely on hash commitments to make partial views verifiable. They operate at different layers and serve different ends.

5. Freshness, revocation, and operational burden

Questions of freshness and revocation are where design choices turn into operational reality. Both systems care about change over time, but they model it differently and distribute the resulting costs differently.

5.1 Key state and continuity in KERI

In KERI, change is central. An identifier is defined by the sequence of key events that control it over time. Rotation, revocation, delegation, and recovery are expressed as explicit events in a key event log [1].

Freshness here is about continuity rather than immediacy. A verifier asks whether the key state it sees forms a valid, uninterrupted chain from inception to the present. Verification can be performed offline, provided the verifier has access to the relevant events and receipts.

Revocation follows the same pattern. An ACDC is revoked through events that change its status within a registry or key state framework [2]. The verifier's task is to reconcile events, not to query a live service.

5.2 Freshness assumptions in Merkle Tree Certificates

Merkle Tree Certificates place freshness at the center of their efficiency model.

If a relying party is up to date on landmarks, certificate validation can be performed with minimal data. If not, validation falls back to heavier proofs that resemble today's certificates [5, 9, 13].

Revocation remains anchored in WebPKI semantics [11]. The Merkle structure optimizes how evidence of issuance and logging is conveyed; it does not replace certificate lifecycle mechanisms.

5.3 Who bears the cost

KERI shifts complexity toward reasoning about event history. In exchange, it relaxes assumptions about connectivity and central availability.

MTCs shift complexity toward distribution and synchronization. They assume that most clients can be kept reasonably current and reward that assumption with reduced on-the-wire cost.

Each approach reflects the constraints of its home ecosystem.

6. Post-quantum readiness

Post-quantum cryptography introduces uneven pressure. Keys and signatures grow larger, and systems already close to their operational limits feel that growth first [4, 14].

Both KERI/ACDCs and Merkle Tree Certificates rely on hash functions as long-lived anchors [10, 14]. Beyond that shared reliance, their responses diverge.

6.1 Merkle Tree Certificates and size pressure

Merkle Tree Certificates address the risk that post-quantum primitives make WebPKI handshakes and logs prohibitively expensive [5].

By amortizing heavy cryptography across many validations and relying on small inclusion proofs in the common case, MTCs reduce the amount of data that must be transmitted during certificate validation [5, 9].

6.2 KERI, ACDCs, and cryptographic agility

KERI emphasizes continuity across algorithm transitions. Key pre-rotation allows identifiers to commit in advance to future key material, enabling algorithm changes without breaking trust relationships [1].

ACDCs inherit this posture. Claims remain bound to identifiers and key state rather than to a fixed signature scheme [2]. Hash commitments preserve identity across cryptographic evolution.

6.3 Different failure modes

Merkle Tree Certificates respond to the risk of unsustainable operational cost at scale. KERI and ACDCs respond to the risk of fractured identity continuity during algorithm transitions.

Both are forms of post-quantum preparedness, but they address different failure modes.

7. Standardization and adoption trajectories

KERI/ACDCs and Merkle Tree Certificates are advancing through different institutional channels and serving different operational constituencies.

7.1 ACDCs and KERI

ACDCs have entered production through multiple paths.

ISO 17442-3 specifies their use for verifiable legal entity identifiers [6]. The GSMA Open Verifiable Calling project uses them to address fraud and impersonation in global voice networks [7]. Within Trust Over IP, the Dossier Task Force defines a composition layer that builds on ACDCs rather than replacing them [8].

These efforts treat ACDCs as a stable substrate rather than as an application protocol.

7.2 Merkle Tree Certificates

Merkle Tree Certificates are being developed as an evolution of WebPKI infrastructure. They are specified as an IETF Internet-Draft and explored through controlled experiments by browser and infrastructure operators [5, 9].

Compatibility with existing trust anchors and fallback paths is a core design requirement.

7.3 Orthogonal adoption paths

ACDCs are moving through identity, legal, and telecom ecosystems. Merkle Tree Certificates are moving through browser and CA infrastructure.

These trajectories are orthogonal rather than competitive.

8. Conclusion

KERI/ACDCs and Merkle Tree Certificates share a vocabulary of hashes and Merkle-like structures. That shared vocabulary can obscure as much as it reveals.

These systems respond to different pressures, operate at different layers, and answer different questions. Graduated disclosure is about shaping how claims are seen. Inclusion proofs are about establishing that claims belong to an auditable history.

Comparing them is useful not because one subsumes the other, but because the contrast sharpens assumptions. A reader who understands those assumptions is better equipped to evaluate new designs without forcing them into familiar but ill-fitting categories.

If this paper succeeds, it leaves the reader with clearer mental models, not a verdict.

References

[1] Smith, S. (2019). *Key Event Receipt Infrastructure (KERI)*. arXiv:1907.02195. https://doi.org/10.48550/arXiv.1907.02195

[2] Trust Over IP KSWG. (2023). *Authentic Chained Data Containers (ACDC) Specification*. Trust Over IP Foundation.

[3] Laurie, B., Langley, A., and Kasper, E. (2013). *Certificate Transparency*. RFC 6962. https://doi.org/10.17487/RFC6962

[4] Bernstein, D. J., et al. (2022). *Post-quantum cryptography*. Communications of the ACM, 65(2). https://doi.org/10.1145/3478437

[5] Ben, D., et al. (2025). *Merkle Tree Certificates*. IETF Internet-Draft, work in progress.

[6] International Organization for Standardization. (2024). *ISO 17442-3:2024 Financial services — Legal entity identifier (LEI) — Part 3: Verifiable LEIs (vLEIs)*. ISO.

[7] GSMA. (2024). *GSMA Foundry launches Open Verifiable Calling project*. GSMA.

[8] Trust Over IP KSWG. (2024). *Dossier Specification*. Draft.

[9] Cloudflare, Inc. (2025). *Bootstrapping Merkle Tree Certificates*. Cloudflare Blog.

[10] Menezes, A., van Oorschot, P., and Vanstone, S. (1996). *Handbook of Applied Cryptography*. CRC Press.

[11] Cooper, D., et al. (2008). *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*. RFC 5280. https://doi.org/10.17487/RFC5280

[12] Hardman, D. (2023). *Verifiable Voice Protocol*. IETF Internet-Draft draft-hardman-verifiable-voice-protocol-02.

[13] Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. https://doi.org/10.17487/RFC8446

[14] National Institute of Standards and Technology. (2024). *FIPS 204: Module-Lattice-Based Digital Signature Standard*. NIST.